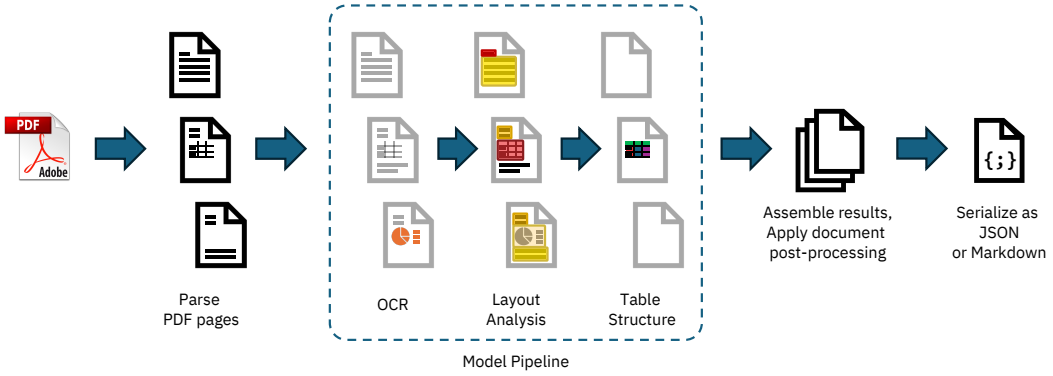


Figure 1: Sketch of Docling's default processing pipeline. The inner part of the model pipeline is easily customizable and extensible.

licensing (e.g. pymupdf [7]), poor speed or unrecoverable quality issues, such as merged text cells across far-apart text tokens or table columns (pypdfium, PyPDF) [15, 14].

We therefore decided to provide multiple backend choices, and additionally open-source a custom-built PDF parser, which is based on the low-level *qpdf* [4] library. It is made available in a separate package named *docling-parse* and powers the default PDF backend in Docling. As an alternative, we provide a PDF backend relying on *pypdfium*, which may be a safe backup choice in certain cases, e.g. if issues are seen with particular font encodings.

3.2 AI models

As part of Docling, we initially release two highly capable AI models to the open-source community, which have been developed and published recently by our team. The first model is a layout analysis model, an accurate object-detector for page elements [13]. The second model is TableFormer [12, 9], a state-of-the-art table structure recognition model. We provide the pre-trained weights (hosted on huggingface) and a separate package for the inference code as *docling-ibm-models*. Both models are also powering the open-access deepsearch-experience, our cloud-native service for knowledge exploration tasks.

Layout Analysis Model

Our layout analysis model is an object-detector which predicts the bounding-boxes and classes of various elements on the image of a given page. Its architecture is derived from RT-DETR [16] and re-trained on DocLayNet [13], our popular human-annotated dataset for document-layout analysis, among other proprietary datasets. For inference, our implementation relies on the *onnxruntime* [5].

The Docling pipeline feeds page images at 72 dpi resolution, which can be processed on a single CPU with sub-second latency. All predicted bounding-box proposals for document elements are post-processed to remove overlapping proposals based on confidence and size, and then intersected with the text tokens in the PDF to group them into meaningful and complete units such as paragraphs, section titles, list items, captions, figures or tables.

Table Structure Recognition

The TableFormer model [12], first published in 2022 and since refined with a custom structure token language [9], is a vision-transformer model for table structure recovery. It can predict the logical row and column structure of a given table based on an input image, and determine which table cells belong to column headers, row headers or the table body. Compared to earlier approaches, TableFormer handles many characteristics of tables, such as partial or no borderlines, empty cells, rows or columns, cell spans and hierarchy both on column-heading or row-heading level, tables with inconsistent indentation or alignment and other complexities. For inference, our implementation relies on *PyTorch* [2].